

Pay and Pray

Portfolio tracker de acciones del mercado de EE. UU.

Informe de Arquitectura, Testing y Rendimiento

Integrantes

Sofía Keceli

Julián Ritondale

Fernando Santisi

Juan Decoud

Juan Luis Martínó

Fecha de entrega: 18/06/2026

Repositorio: RugPull-Inc / pay-and-pray

1. Introducción y enfoque de calidad

Pay and Pray es un portfolio tracker de acciones cotizadas en mercados de Estados Unidos, disponible como aplicación web y mobile sobre una misma API. El usuario puede registrar operaciones de compra y venta, consultar el valor y la ganancia/pérdida de sus posiciones, seguir empresas en una watchlist y acceder a información financiera oficial. El sistema se apoya en dos fuentes de datos externas reales: la API pública EDGAR de la SEC para los datos financieros, y Yahoo Finance para los precios de mercado.

Ahora bien, el producto en sí fue el medio y no el fin. La consigna pedía un sistema deliberadamente minimalista, y el objetivo real del trabajo era poner en práctica los mecanismos de aseguramiento de la calidad sobre un caso con suficiente complejidad de dominio (cálculo de P&L, integraciones externas inestables, rate limiting) como para que esas técnicas tuvieran sentido. Por eso, todo el informe está organizado alrededor de una pregunta: qué decidimos y por qué, más que qué construimos.

1.1 Testeabilidad como requisito de diseño

La decisión que más condicionó al resto fue tratar la testeabilidad como un requisito de diseño, y no como una consecuencia. En la práctica esto se tradujo en reglas que aplicamos de forma sistemática en el backend: cada capa se expone detrás de una interfaz (`Service/ServiceImpl`, `Repository/RepositoryImpl`) para poder reemplazar dependencias en los tests; el acceso a datos vive aislado, de modo que una única clase conoce JPA y el resto habla contra una interfaz de dominio; y cada integración externa se define como una interfaz con los métodos justos, con una sola implementación que maneja HTTP, rate limiting y errores. Así, la lógica de negocio se prueba sin levantar Spring ni la base, y las integraciones se pueden fakear sin depender del servicio real.

1.2 Cómo organizamos el avance

El trabajo se estructuró en seis features, cada una con sus user stories y criterios de aceptación. Avanzamos feature por feature, y dentro de cada una construimos de la persistencia hacia afuera: primero el modelo de datos y la API, después la interfaz web que la consume. Los tests unitarios y de integración no fueron un paso posterior, sino parte de construir cada feature; una historia no se daba por terminada sin esa cobertura. Priorizamos primero las piezas de mayor riesgo —las integraciones con EDGAR y Yahoo Finance, por ser externas e inestables— para validarlas temprano. Los tests end-to-end (Cypress y Appium) y las pruebas de carga con Locust llegaron sobre flujos ya estables, porque ejercitan el sistema integrado y no funcionalidad en construcción.

2. Arquitectura

El sistema se divide en cuatro componentes que corren como procesos separados y se orquestan con Docker Compose: una API REST, un frontend web que además se empaqueta como app Android, un proceso batch de precios y una base de datos PostgreSQL. La idea central del diseño es separar responsabilidades por proceso y dejar que cada uno escale y falle de forma independiente, compartiendo únicamente la base de datos como fuente de verdad.

2.1 Decisiones de stack

- **Kotlin 2.1.21 + Spring Boot 3.4.5** para la API REST. Es un stack con tipado fuerte, null-safety y un ecosistema maduro para JPA, validación, seguridad con JWT y caché. Spring nos da la inyección de dependencias y un modelo de capas (controller, service, repository) que ya teníamos estandarizado en las guías del proyecto. Corre sobre Java 21.
- **React 19 + Vite 7 + TypeScript + Tailwind** para el frontend web. React por ser el estándar del equipo, Vite por su velocidad de build y dev server, TypeScript por seguridad de tipos y Tailwind para estilos. Toda la lógica de UI, ruteo y servicios de API vive en un único codebase.
- **Capacitor 8** envuelve el mismo build web para generar la app Android, en lugar de reescribir la UI en nativo. El detalle se desarrolla en la sección Mobile.
- **Python 3.12** para el batch de precios. Python encaja por la disponibilidad de librerías de datos de mercado (yfinance) y por aislar una tarea de naturaleza distinta a la API transaccional. Usa psycpg2 para escribir en la base y pandas para procesar.
- **PostgreSQL 16** como base de datos única y compartida. PostgreSQL por madurez, soporte transaccional y buena integración con Flyway para migraciones versionadas.

2.2 Separación API / batch

La API y el batch de precios son dos servicios distintos, cada uno en su contenedor. La API atiende las operaciones transaccionales del usuario (registro, login, búsqueda de empresas, compra y venta, portfolio con P&L). El batch se ocupa de una tarea de fondo diferente: recolectar todos los tickers seguidos (posiciones, watchlist y tracked tickers), pedir precios a Yahoo Finance, hacer upsert en la tabla de precios y registrar el estado de cada corrida en `batch_runs`.

Los dos procesos no comparten código ni memoria: se comunican por la base de datos y por un disparo HTTP. La API invoca al batch con un POST a `/trigger` del servicio de precios (clase `HttpBatchTriggerService`, URL configurable con `BATCH_SERVICE_URL`), y además hay un refresh de precios opcional al arranque. Esta separación permite que un trabajo de precios pesado o lento no bloquee las peticiones de los usuarios, y que cada servicio se reinicie o escale por su cuenta.

2.3 Orquestación con Docker Compose

Docker Compose levanta los cuatro servicios con sus dependencias y healthchecks. El backend espera a que la base y el batch estén sanos antes de arrancar; el batch espera a la base; el frontend espera al backend. La base persiste en un volumen nombrado (`postgres_data`).

Servicio	Imagen / build	Puerto	Depende de
----------	----------------	--------	------------

db	postgres:16-alpine	5432	—
backend (API)	build ./backend (Java 21)	8080	db, price-batch
price-batch	build ./price-batch (Python 3.12)	8081	db
frontend	build ./frontend	3000	backend

El backend usa build multi-stage (compila con JDK, corre con JRE) y arranca con ddl-auto en validate, de modo que Hibernate nunca modifica el esquema: los cambios de base se aplican solo por migraciones Flyway versionadas.

2.4 Seguridad y manejo de errores

La autenticación se resuelve con JWT y sesiones stateless: un filtro valida el token en cada request antes de llegar al controller. La regla por defecto en `SecurityConfig` es denegar —todo endpoint requiere autenticación— salvo los explícitamente liberados (registro y login, consulta pública de empresas, disparo del batch y consulta de última actualización de precios). La identidad del usuario nunca viene del request: el ID se toma del `Authentication` que inyecta Spring a partir del token, evitando que un usuario opere sobre el portfolio de otro.

Cada modo de falla esperado tiene su propia excepción no chequeada (email duplicado, recurso inexistente, fondos/cantidad insuficiente, EDGAR no disponible). Todas se registran en un único `GlobalExceptionHandler` que las mapea al código HTTP correcto (409, 404, 503, 400) y devuelve siempre la misma forma de respuesta. El beneficio es un contrato de error uniforme para web y mobile, y lógica de negocio que puede fallar lanzando una excepción sin preocuparse por el formato HTTP.

3. Features y user stories

La especificación del sistema se organizó en tres niveles: **features** (las seis grandes capacidades del producto), **user stories** (las historias concretas dentro de cada feature, desde la perspectiva del usuario) y **escenarios de aceptación** (las condiciones puntuales que debían cumplirse). Los escenarios fueron la unidad operativa: funcionaron como Definition of Done y, en buena medida, como el guion de los tests. Por ejemplo, la user story "Vender acciones" no se consideraba terminada por permitir vender, sino cuando se verificaba —entre otros— que el sistema no permitiera vender más unidades de las disponibles, y existía una prueba que lo confirmaba.

- **Feature 1 — Gestión de cuenta de usuario.** Registro con email y contraseña, login y acceso autenticado. Rechaza emails inválidos, contraseñas vacías y emails duplicados (409), y no expone información sensible en los errores de login.
- **Feature 2 — Búsqueda y consulta de empresas (SEC EDGAR).** Búsqueda por nombre o ticker, métricas financieras clave (Revenue, Net Income, EPS, Total Assets, Total Liabilities), evolución histórica de 4 a 8 quarters y filings recientes (10-K, 10-Q).
- **Feature 3 — Proceso de actualización de precios (Yahoo Finance).** Disparo manual o por CI del batch, actualización de precios almacenados y registro de fecha/hora; una corrida fallida no sobrescribe la última fecha exitosa.
- **Feature 4 — Gestión de portfolio.** Compra y venta, consulta de posiciones con P&L, valor total e historial de operaciones. Toda valorización se calcula contra el último precio almacenado; no se permite vender más de lo disponible.
- **Feature 5 — Watchlist y comparación de empresas.** Seguir empresas sin posición abierta (sin duplicados) y comparar sus métricas financieras clave, indicando si el usuario tiene posición en cada una.
- **Feature 6 — Experiencia móvil y rendimiento bajo carga.** Agrupa los requisitos no funcionales: usabilidad desde el celular en todas las pantallas principales (6.1–6.6) y comportamiento bajo carga —conurrencia, tiempos de respuesta, estabilidad sostenida y degradación controlada— (6.7–6.10).

4. Estrategia de testing

La estrategia sigue la pirámide de tests: muchos tests unitarios rápidos en la base, una capa intermedia de tests de integración que ejercitan HTTP y base de datos reales, y una punta angosta de tests end to end (web y mobile). Por fuera de la pirámide funcional, una capa de carga y estrés valida el comportamiento bajo presión. La regla de oro es: cuanto más abajo, más rápido y más barato el test, así que ahí concentramos el volumen.

4.1 Tests unitarios (base de la pirámide)

Alrededor de 34 clases de test unitario, sin levantar contexto de Spring. En vez de Mockito se usan clases Fake escritas a mano (unas 13) que implementan las interfaces de servicio y repositorio, con flags de comportamiento y contadores cuando hace falta verificar interacciones. Por ejemplo `FakeTokenService`, `FakeUserService`, `FakePriceService` o `FakeTickerEdgarClient`. Cubren lógica de negocio pura: login, compra, venta, cálculo de transacciones, caché de tickers y manejo de errores de la API de EDGAR. Al no usar Spring, corren en milisegundos.

4.2 Tests de integración (capa media)

Doce clases que extienden `IntegrationTestBase`, la cual combina `SpringBootTest`, `MockMvc` y un contenedor PostgreSQL 16 real levantado con Testcontainers. Prueban el request y response HTTP completo: códigos de estado, forma del JSON y persistencia real en la base. La clase base ofrece `loginAndGetToken()` para obtener un JWT válido y golpear endpoints protegidos, y limpia las cachés entre tests. Cubren auth, usuarios, empresas (incluido el escenario de EDGAR caído), compra, venta, vista de portfolio, historial, watchlist, comparación y precios.

4.3 End to end web con Cypress

Nueve especificaciones de Cypress, divididas a propósito en dos modos. Los tests interceptados usan `cy.intercept` para stubear la API y verificar contratos de UI y caminos de error de forma rápida y determinística (watchlist, portfolio-trade, portfolio-history, portfolio-view). Los tests reales golpean el backend de verdad de punta a punta (auth, company-browse, company-detail, watchlist-real), incluyendo registro, login, búsqueda, compra y comparación con datos reales. La combinación da velocidad donde importa y confianza de extremo a extremo donde hace falta.

4.4 End to end mobile con Appium

Tres especificaciones que corren la app Android empaquetada con Capacitor, usando Appium con el driver UiAutomator2 y WebdriverIO como runner. Cubren los flujos de watchlist (agregar, ver, comparar), auth y portfolio (registro, login, precios, compra, venta, historial) y detalle de empresa. Corren tanto en emulador como en celular físico por USB.

4.5 Carga y estrés con Locust

Por fuera de la pirámide funcional, Locust ejercita cinco workflows (registro más login, búsqueda, ver portfolio, comprar, y vender sin poseer que debe dar 4xx). Sirve para hallar la capacidad, validar tiempos bajo carga, detectar fugas en corridas largas y verificar la degradación controlada. Los resultados están en la sección siguiente.

Capa	Herramienta	Cantidad	Qué valida
Unitarios	Fake* a mano (sin Spring)	~34	Lógica de negocio aislada
Integración	MockMvc + Testcontainers	12	HTTP + base de datos reales
E2E web	Cypress (intercept + real)	9	Flujos de usuario en navegador
E2E mobile	Appium + UiAutomator2	3	Flujos en Android real
Carga / estrés	Locust	5 workflows	Rendimiento y degradación

5. Resultados de carga

Todas las corridas son del 18/06/2026 sobre un entorno fijo y reproducible para que los números sean comparables. Las decisiones de medición se mantuvieron constantes:

- **Entorno:** backend limitado a 2 cores (override `docker-compose.stress.yml`). Un techo de CPU reproducible que además permite provocar el quiebre; con 8 cores la app es tan liviana que no rompe.
- **Auth real:** cada usuario virtual se registra y loguea con su propia cuenta (sin pool de tokens). Esto es 100% representativo y evita contención artificial.
- **Think-time:** 0 en todo. En load queda por debajo de la capacidad; en stress, muy por encima.
- **Timeout:** 30 s por request. Un request colgado cuenta como fallo explícito, no como latencia ilegible.

El orden de corridas fue: capacity (hallar el knee), load, soak, dos modos de stress y recovery. Se reportan p50, p95 y p99, nunca el promedio, porque el promedio esconde la cola.

5.1 Tabla de resultados

Escenario	Usuarios	p50	p95	p99	Throughput	5xx	Estado
Capacity (knee)	~500	710 ms	760 ms	—	~1.950 rps	0	OK
Load (10 min)	300	77 ms	560 ms	910 ms	~1.836 rps	0	OK 6.7/6.8
Soak (30 min)	300	73 ms	410 ms	760 ms	~1.762 rps	0	OK 6.9
Stress-A shock onboarding	rampa +1.000	—	30 s+ (techo)	—	colapsa	0	OK 6.10
Stress-B carga gradual	rampa +150	—	230 ms a 12 s	—	~2.000 rps	0	OK 6.10

El único fallo del soak fue 1 login 401 en 2,7 millones de requests (race transitorio en el arranque del usuario, no un 5xx). En los dos modos de stress, todos los fallos posteriores al quiebre son a nivel de conexión (connection reset, disconnect o timeout de 30 s): cero 5xx y cero corrupción de datos.

5.2 Dimensionamiento derivado

- **Capacity:** el RPS se aplanan en ~1.950 rps alrededor de 500 usuarios (CPU a 2 cores saturada). Por encima de ese punto solo crece la latencia. El knee queda en ~500 usuarios.
- **Carga de operación normal:** se fijó en 60% del knee = 300 usuarios. Zona cómoda, con RPS aún sin saturar y p95 alrededor de 560 ms.
- **Umbral de p95 (US 6.8):** 1.000 ms (el p95 medido en load fue ~560 ms, más margen). El error rate aceptable es menos de 1% de 5xx; los 4xx esperados del workflow de venta sin poseer no

cuentan.

5.3 Conclusiones por user story

- **6.7 Concurrencia:** 300 usuarios, 10 minutos, 0 errores 5xx en 1.101.723 requests.
- **6.8 Tiempos bajo carga:** p95 agregado de 560 ms, por debajo del umbral de 1.000 ms (search ~110 ms; portfolio, buy y sell ~680 ms).
- **6.9 Estabilidad sostenida:** 30 minutos con p95 plano en 520 a 580 ms sin creep, memoria estable en 1,01 a 1,05 GiB (sin fugas) y CPU constante en ~205%.
- **6.10 Degradación controlada:** validada en dos modos. Stress-A (shock de onboarding, +1.000 por escalón) quiebra a ~1.050 usuarios y degrada descartando conexiones y devolviendo timeouts explícitos, sin 5xx ni crash, y se recupera al bajar la carga (a 150 usuarios vuelve a 0 errores y p95 230 ms). Stress-B (carga gradual, +150 por escalón) aguanta 0 errores hasta 8.400 usuarios; la degradación es 100% latencia (p95 lineal de 230 ms a 12 s) con throughput plano en ~2.000 rps, y el breakpoint a 8.450 usuarios lo marca el límite de conexiones de Tomcat (max-connections 8.192), no un error de la app.

Hallazgo principal: comparando los dos modos de stress (misma base, mismos workflows, mismos 2 cores, la única diferencia es la tasa de onboarding), el cuello de botella bajo presión no son los workflows ni el volumen de datos, sino el bcrypt del registro y login. Meter ~1.000 altas de golpe satura los cores. Con onboarding gradual el sistema aguanta hasta el techo de conexiones de Tomcat.

6. EDGAR: rate limiting, caché e impacto en stress

Los datos de empresas (listado de tickers y métricas financieras) vienen de la API pública de SEC EDGAR. La integración está encapsulada detrás de la interfaz `EdgarClient`, con una única implementación (`EdgarApiClient`) que maneja HTTP, límite de tasa y envoltura de errores, lo que la hace fácil de fakear en tests.

6.1 Rate limiting

SEC EDGAR exige un header User-Agent válido que identifique a la aplicación (se inyecta por variable de entorno `EDGAR_USER_AGENT`) y limita a 10 requests por segundo. El cliente respeta ese límite con un rate limiter `Bucket4j` configurado en capacidad 10 con recarga de 10 por segundo: cada llamada consume un token de forma bloqueante antes de salir a la red, de modo que nunca superamos el límite de SEC. La búsqueda full-text se restringe a formularios 10-K, una decisión consciente para un portfolio tracker: todas las grandes empresas públicas presentan 10-K, así que el resultado es relevante.

6.2 Estrategia de caché

Sobre el rate limiting se apoya una caché en memoria con `Caffeine` para no repetir llamadas a EDGAR:

Caché	TTL	Tamaño	Contenido
companyDetails	1 hora	10.000 entradas	Métricas financieras por empresa (CIK)
companyTickers	24 horas	listado completo	Listado de tickers de EDGAR

Además hay un fallback empaquetado para cuando EDGAR bloquea el archivo `company_tickers.json` (responde 403). Si está habilitado (`EDGAR_TICKERS_FALLBACK_ENABLED`) y la llamada en vivo falla, el sistema carga un JSON bundled de más de 50 empresas grandes (AAPL, MSFT, TSLA, NVDA, AMZN y otras) en lugar de caerse. Si el fallback está deshabilitado, la excepción se propaga. Esto mantiene la búsqueda de empresas funcionando aun cuando SEC nos rechaza.

6.3 Impacto en el stress testing

El rate limit de 10 req/s de EDGAR podría parecer un cuello de botella para las pruebas de carga, pero por diseño no lo es. Antes de medir, el script `warmup.py` precalienta las cachés: dispara un refresh de precios y hace una única llamada que deja caliente el ticker cache (válido por 24 horas).

Durante la corrida, las búsquedas usan prefijos de tickers que pegan contra el ticker cache, no contra EDGAR en vivo. Por eso, en las corridas de stress, EDGAR nunca aparece como el limitante: el quiebre lo marcan la CPU (bcrypt del onboarding) y el límite de conexiones de Tomcat, no el rate limit de SEC. Además, el proceso de actualización de precios contra Yahoo corre una única vez por invocación y no forma parte del bucle de carga, así que tampoco genera tráfico continuo sobre el servicio externo. Esto fue intencional, para medir la capacidad de nuestra aplicación y no la de un servicio externo.

7. Mobile

7.1 Decisión: Capacitor en lugar de nativo

Se eligió Capacitor para empaquetar como app Android el mismo build web de React en lugar de escribir una app nativa nueva. Las razones:

- **Un solo codebase:** el frontend ya es una app React basada en navegador y todavía no necesita una UI nativa propia. Capacitor envuelve el build de Vite (carpeta dist) dentro de un shell Android.
- **Una sola fuente de verdad:** comportamiento de producto, ruteo, servicios de API, estilos y tests quedan centrados en un único frontend, en vez de duplicarse en una base nativa aparte.
- **Menos fricción:** mantener `android/` dentro de `frontend/` es la estructura por defecto de Capacitor y reduce complejidad de configuración. Un paquete `mobile/` separado pediría cableado extra para consumir el build del frontend, sin beneficio hoy.

7.2 Cómo funciona

El identificador de la app es `com.payandpray.app`. El flujo de build es: Vite compila la app React a `frontend/dist`, Capacitor sincroniza ese dist dentro del proyecto Android, y Gradle compila el APK con los assets web embebidos. Los comandos quedan envueltos en scripts npm (`mobile:build`, `mobile:android`, `cap:open:android`).

7.3 Limitaciones encontradas

- **Red y localhost:** la app mobile no resuelve localhost como lo hace el navegador de escritorio. En el emulador hay que apuntar `VITE_API_URL` a `http://10.0.2.2:8080` (alias de la máquina host desde dentro del emulador), y el CORS del backend debe permitir el origen de Capacitor (normalmente `https://localhost`).
- **Dispositivo físico:** para correr contra un celular físico por USB hay que tunelear el puerto con `adb reverse tcp:8080 tcp:8080` para que el dispositivo vea el backend en localhost.
- **Versión de JDK:** el build Android de Gradle requiere JDK 17 o 21. JDKs más nuevos (por ejemplo Java 25) rompen el build, así que hay que fijar `JAVA_HOME` a una versión soportada.
- **E2E:** los tests E2E mobile dependen de tener el backend corriendo con un precio de AAPL disponible y del APK debug compilado; los timeouts de instalación y de arranque de UiAutomator2 se subieron para tolerar emuladores lentos.

8. CI/CD, versionado y flujo de Git

La integración continua se resuelve con GitHub Actions, en un workflow versionado en el propio repositorio (`.github/workflows/ci.yml`). Se dispara en cada push y cada pull request hacia las ramas `main` y `dev`, de modo que ningún cambio llega a las ramas principales sin pasar por validación automática.

8.1 Pipeline con jobs selectivos

El pipeline arranca con un job que usa path filtering para detectar qué parte del repositorio cambió y decidir qué jobs correr, acortando el feedback sin perder cobertura. El job **backend** verifica formato (`ktlintCheck`), compila y corre la suite de tests (unitarios e integración con Testcontainers). El job **frontend** chequea formato (Prettier), lintea (ESLint), compila y corre los tests. El job **e2e** levanta el stack completo con Docker Compose y corre Cypress contra él; está acotado a pull requests hacia `main` por ser el más pesado, y sube las capturas de Cypress como artefacto ante una falla.

8.2 Imágenes Docker reproducibles

Cada componente tiene su propio Dockerfile; los del backend y el frontend son multi-stage (una etapa compila, una etapa final liviana lleva solo el artefacto ejecutable). El resultado son imágenes reproducibles que `docker compose` ensambla en el stack completo, de modo que el sistema se construye y levanta sin pasos manuales de instalación.

8.3 Versionado con SemVer

El avance se versionó en GitHub siguiendo Versionado Semántico. Las releases se publicaron en hitos: cada feature nueva incrementó la versión MINOR y cada corrección la PATCH, dando una historia trazable.

Versión	Hito
v1.0.0	Feature 1 — Gestión de cuenta de usuario
v1.1.0	Feature 2 — Búsqueda y consulta de empresas
v1.2.0	Feature 3 — Proceso de actualización de precios
v1.3.0	Feature 4 — Gestión de portfolio
v1.4.0	Feature 5 — Watchlist y comparación de empresas
v1.5.0	Feature 6 — Experiencia móvil y rendimiento bajo carga

Entre medio se publicaron releases PATCH (v1.2.1, v1.3.1, v1.4.1) para correcciones puntuales.

8.4 Flujo de Git y trazabilidad

El trabajo en equipo se ordenó alrededor de un flujo de Git con foco en la trazabilidad de punta a punta, desde la especificación hasta el código integrado:

- **Issues derivados de los docs.** Cada unidad de trabajo nace de un issue, y los issues salen directamente de las user stories ya definidas en la documentación. Una user story se convierte en

uno o más issues concretos.

- **Una rama por issue.** Cada issue se desarrolla en su propia rama de trabajo, nombrada con el número de issue y la feature/US (por ejemplo `54-f4-us4-backend-historial-de-operaciones`). Son ramas cortas, de vida menor a una semana —en general un par de días— para integrar seguido y evitar divergencias grandes.
- **Conventional Commits.** Los commits siguen la convención (`feat:`, `fix:`, `test:`, etc.), con descripción donde aporta contexto, lo que hace legible la historia y habilita el versionado por tipo de cambio.
- **Pull requests hacia `dev`.** Cada rama de trabajo se integra mediante un PR hacia `dev` (y `dev` hacia `main` en los hitos). El PR es el punto de corrección: pasa por peer review de otro integrante y por el CI, que corre el testing automatizado como compuerta antes de poder mergear.
- **GitHub Projects.** Se usó un tablero que asocia issues, PRs y commits, dando trazabilidad completa: desde una user story se puede llegar al issue, a la rama, a los commits y al PR que la cerró. Los issues avanzan por estados —Backlog, Todo, In Progress, Review, Done— que reflejan en qué punto del flujo está cada unidad de trabajo.

9. Decisiones de diseño destacadas

9.1 Valorización contra el precio almacenado

Es la decisión transversal del sistema: ningún cálculo de negocio consulta Yahoo Finance en vivo. Las compras, ventas y la valorización del portfolio usan el último precio almacenado (`PriceService.getLatestPrice`). Esto hace la operación determinística, rápida (sin latencia de red en el camino crítico) y testeable (basta sembrar un precio para probar el flujo). El precio se actualiza por fuera, mediante el batch.

9.2 Posiciones por costo promedio ponderado

Combinar múltiples compras y ventas de un mismo ticker en una posición coherente fue la parte menos trivial del dominio. Cada posición se modela con dos datos —cantidad y precio de compra promedio— mantenidos por costo promedio ponderado: al comprar, el nuevo promedio pondera la posición existente con la nueva compra; al vender, solo se reduce la cantidad y el promedio no cambia (si llega a cero, la posición se elimina). El P&L no realizado de cada posición es $(\text{precioActual} - \text{avgBuyPrice}) \times \text{cantidad}$. Se eligió costo promedio y no lotes FIFO por proporcionalidad: es el modelo más simple que responde correctamente lo que el producto necesita, sin la complejidad de rastrear lotes individuales. Cada operación se persiste igual como una transacción independiente, de modo que el historial completo queda registrado.

9.3 Aritmética con `BigDecimal` y datos faltantes

Por tratarse de dinero, todos los cálculos monetarios usan `BigDecimal` (nunca `Double`), con redondeo explícito (`HALF_UP`) y escala fija donde corresponde, evitando errores de coma flotante que harían diferir dos caminos de cálculo equivalentes. Además, si un ticker no tiene precio almacenado, la posición no rompe el cálculo: se muestra con su cantidad y precio de compra, con valor actual y P&L en nulo, y simplemente no suma al valor total, en lugar de devolver un error.

10. Posibles puntos de mejora

El trabajo cumplió sus objetivos, pero el proceso dejó a la vista varios puntos donde, con más tiempo o en una próxima iteración, habríamos hecho las cosas distinto.

- **Definir el pipeline Features → User Stories → Escenarios desde el principio.** Los escenarios de aceptación se incorporaron en buena medida sobre la marcha. Escribirlos antes de implementar, como contrato previo, habría reforzado el enfoque ATDD y vuelto más natural escribir el test antes que el código.
- **Refinar el `CLAUDE.md` para asegurar calidad por defecto.** Varias convenciones del proyecto viven en ese documento. Con más tiempo, valdría la pena plantear algunas de esas garantías de forma más completa y precisa, para que se apliquen por defecto sin tener que explicitar cada detalle en el momento, y el resultado dependa menos de recordatorios puntuales.
- **Releases más granulares y lineales.** El versionado mapeó cada feature a una release, pero fueron releases gruesas. Releases más finas —por ejemplo por user story terminada— darían una historia de versiones más precisa y acercarían el proyecto a un esquema de entrega continua.
- **Mejor solución para los tracked tickers.** Hoy el conjunto de tickers seguidos funciona como una whitelist sembrada, lo que resuelve el problema técnico pero limita la experiencia: el usuario no puede operar libremente cualquier ticker. Habría valido la pena explorar una mejor solución de UX, por ejemplo incorporar un ticker bajo demanda la primera vez que un usuario lo busca u opera, en lugar de depender de una lista predefinida.

11. Conclusión

El objetivo del trabajo no era tanto construir un portfolio tracker como usar ese producto para aplicar aseguramiento de la calidad de software, y ese fue el hilo que recorrió todas las decisiones. La tesis que adoptamos desde el principio —tratar la testeabilidad como un requisito de diseño— se materializó en una arquitectura en capas con interfaces, integraciones externas encapsuladas y fakeables, y un esquema de persistencia versionado; y se pagó después en una pirámide de tests que es el reflejo de esa estructura, no un agregado posterior.

Sobre esa base, cumplimos los requisitos de la consigna: las seis features con sus user stories y escenarios de aceptación, la integración real con SEC EDGAR (con su rate limiting y caché) y con Yahoo Finance a través de un batch independiente, el diseño de precio almacenado que hace determinística la valorización, los tests unitarios con Fake*, los de integración con Testcontainers, los end-to-end en web (Cypress) y mobile (Appium), las pruebas de carga y estrés con Locust diferenciando ambos esquemas, el stack completo en Docker Compose con volúmenes persistentes, el versionado SemVer y la integración continua en GitHub Actions.

Más allá del cumplimiento, lo más valioso fueron los lugares donde la forma de trabajar dejó aprendizajes: el orden en que conviene definir features, historias y escenarios; cómo el contraste entre dos esquemas de stress reveló que el cuello bajo onboarding masivo era el auth y no los flujos de negocio; y los puntos de mejora que identificamos para una próxima iteración. El sistema quedó funcionando, probado de punta a punta y con su calidad documentada, que era exactamente lo que el trabajo se proponía demostrar.